flora 코드 리뷰

1. 프로젝트 개요

- 한 줄 설명: FLORA v1 — 1인 창작자가 AI로 전자책·사진집·룩북·오디오북을 기획부터 PDF/ePUB/음성 파일까지 만드는 "1인 출판 AI SaaS"의 첫 구현본 (CLAUDE.md, PRD.md 1.0, 2026-06-14).
- 목적/대상 사용자: 원고는 있지만 편집·출판 경험이 없는 1인 작가, 사진 작가, 인플루언서, 소규모 독립 출판사.
- 저장소 상태: platform 저장소에 아직 커밋되지 않은 untracked 폴더. .gitignore가 .env*를 제외하고 있으나, 가상환경(api/.venv311, f5-tts/venv)·모델 체크포인트(f5-tts/ckpts)·빌드 산출물(frontend/tsconfig.tsbuildinfo, frontend/-w, api/server.log)이 소스 트리에 섞여 있다.

기술 스택

레이어	기술	근거
프론트	Next.js 16.2.6 (App Router, src/proxy.ts 미들웨어), React 19.2, TypeScript, Tailwind v4, Shadcn v4, @xyflow/react (캔버스), react-dropzone, sonner	frontend/package.json
백엔드	FastAPI 0.115, Uvicorn, Pydantic v2, python-jose, supabase-py 2.10	api/requirements.txt
DB/인증	Supabase (PostgreSQL + Auth, RLS), 마이그레이션 SQL 4개	supabase/migrations/001~004
AI(텍스트)	Anthropic / OpenAI / Google Gemini 직접 호출 + OpenRouter 폴백 (llama, mistral 포함)	api/routers/llm.py:67-205
AI(이미지)	DALL-E 3, Gemini Imagen 3, Together FLUX, xAI Grok, Recraft, OpenRouter 이미지	api/routers/cover.py:95-268
TTS	F5-TTS 로컬 서버(포트 9000, 저장소 내 벤더링), FastAPI 래퍼 f5-tts/api_server.py	api/routers/tts.py, f5-tts/api_server.py
PDF/ePUB	WeasyPrint(1차) -> fpdf2(폴백), Typst CLI, 직접 조립한 ePUB3 zip, Word 호환 HTML(.doc)	api/routers/export.py
결제	Stripe Checkout + Webhook (키 없으면 mock)	api/routers/billing.py
인프라	Vercel(프론트), Railway(railway.toml) 또는 Render(API), GitHub Actions CI/CD	railway.toml, .github/workflows/*.yml

현재 완성도 추정: 약 55%

- 근거(완료): 전자책 11단계 파이프라인의 LLM 엔드포인트(기획안·목차·섹션 집필·메타데이터)가 실제로 호출되고 200을 반환한 로그가 있다(api/server.log). WeasyPrint/fpdf2/Typst 3중 PDF 경로, ePUB 빌더, 7종 이미지 제공자 표지 생성, F5-TTS 래퍼와 멀티보이스 합성, 60인 페르소나 데이터(frontend/src/lib/personas.ts 620줄), 5대 판형 프리셋(frontend/src/lib/print-presets.ts 714줄)이 모두 구현되어 있다.
- 근거(미완): 백엔드에 플랜/인증 강제가 거의 없고(LLM·TTS·내보내기 전부 무인증), Stripe 웹훅 서명 검증이 없으며, 사진집·룩북 페이지는 128~296줄의 얇은 셸이다. 배포 설정(Dockerfile·deploy.yml·railway.toml)은 실제로 동작하지 않을 가능성이 높다. 프론트 new/ebook/page.tsx가 4,095줄 단일 컴포넌트로 유지보수 한계에 도달했다.

핵심 기능 목록

- 전자책: 장르/분량 -> 페르소나 60인 선택 -> AI 기획안 3종 -> 어조 슬라이더 -> AI 목차 3종 -> 챕터/소제목 편집 -> 섹션 집필(옴니버스 모드 포함) -> 이어쓰기/살붙이기/다듬기 -> AI 표지 -> Multi-LLM 고도화 -> 판형 설정 후 PDF/ePUB/Typst/DOC 내보내기
- 원고 업로드 경로(projects/new/ebook/manuscript), 사진집/룩북 셸, React Flow AI 캔버스(Pro)

- 오디오북: 7종 보이스 프리셋, 등장인물 추출(정규식), 레퍼런스 WAV 업로드, 멀티보이스 합성
- 결제: Free/건당/Basic/Pro/Business 5단계 플랜, Stripe Checkout, 무료 체험
- 대기자 명단(in-memory), 설정(선호 LLM, API 키 — 로컬 저장)

2. 구조 분석

경로	역할	비고
api/main.py (41줄)	FastAPI 앱, CORS, 9개 라우터 등록	waitlist를 /api prefix로 중복 등록
api/db.py (182줄)	Supabase/in-memory 이중 DB 레이어, 플랜 마스터, 사용자 설정	_USE_SUPABASE를 import 시점에 평가
api/routers/auth.py (61줄)	verify_token — mock -> Supabase SDK get_user -> HS256 폴백	요청마다 create_client
api/routers/cover.py (369줄)	optional_verify(auth 중복 구현) + 7종 이미지 제공자	billing.py가 여기서 auth를 import
api/routers/llm.py (1,759줄)	call_llm 다중 제공자 라우팅, 10개 LLM 엔드포인트, 대량 mock 데이터	인증 없음, 동기 SDK 호출
api/routers/export.py (527줄)	ePUB/PDF(WeasyPrint->fpdf2)/html-to-pdf/Typst/DOC	인증 없음, SSRF 가능
api/routers/tts.py (223줄)	F5-TTS 프록시, 멀티보이스 합성, 레퍼런스 업로드	인증 없음, 경로 조작 가능
api/routers/billing.py (165줄)	Stripe checkout/webhook/plan/status/free-trial	웹훅 서명 미검증
api/routers/projects.py, settings.py, waitlist.py	CRUD, 설정, 대기자	waitlist 경로 버그
f5-tts/api_server.py (158줄)	F5-TTS /v1/speech 래퍼, torchaudio soundfile 패치	동기 추론, 임의 경로 읽기
f5-tts/src/f5_tts/ 하위 전체	업스트림 F5-TTS 전체(학습·평가·Triton 런타임 포함) 벤더링	저장소 비대화
frontend/src/app/(dashboard)/dashboard/projects/new/ebook/page.tsx (4,095줄)	전자책 11단계 위저드 전체	useState 68개, fetch 12곳
frontend/src/app/.../new/ebook/manuscript/page.tsx (1,422줄)	원고 업로드 경로 위저드	ebook 페이지 로직 상당 부분 복제
frontend/src/components/project/chapter-wizard.tsx (1,473줄), weasyprint-editor-panel.tsx (1,067줄), persona-card-grid.tsx (739줄)	대형 컴포넌트	
frontend/src/app/api/books/ 하위, api/export/ 하위	Next Route Handler: merge / transform / build-html / build-typst / pdf / typst 프록시	서버사이드 HTML 조립
frontend/src/lib/api.ts (130줄)	projects/cover/billing/settings 클라이언트	LLM 호출은 미포함(raw fetch 산재)
frontend/src/hooks/use-is-pro.ts, lib/admin.ts	Pro 판별(localStorage -> admin 이메일 -> DB)	비밀번호 "1234" 번들 포함
supabase/migrations/001~004	projects/project_files/user_settings/user_plans/plans, API 키 컬럼	user_plans RLS 과다 허용
railway.toml, api/Dockerfile, .github/workflows/ci.yml, deploy.yml, start-dev.ps1	배포·개발 스크립트	여러 설정 불일치

데이터 흐름 요약

브라우저(Next.js 클라이언트 컴포넌트)가 Supabase 세션의 access_token을 꺼내 FastAPI(NEXT_PUBLIC_API_URL)로 직접 호출한다. 프로젝트 CRUD만 Bearer 토큰을 붙이고(frontend/src/lib/api.ts:42-70), LLM·TTS·내보내기 호출은 토큰 없이 raw fetch로 호출한다(new/ebook/page.tsx:428-1017, audiobook/page.tsx:168-273). 전자책 작업 상태(기획안·목차·집필 결과·표지)는 localStorage 초안(hooks/use-draft.ts)에만 저장되며, 최종 원고만 projects.content에 PATCH로 저장된다. 출판 단계에서는 Next Route Handler /api/books/build-html이 판형 프리셋과 템플릿으로 완성 HTML을 조립하고, /api/export/pdf가 WeasyPrint 서버(WEASYPRINT_SERVER_URL) 또는 FastAPI /api/export/pdf로 전달한다. 오디오북은 FastAPI가 F5-TTS 서버 /v1/speech에 세그먼트 단위로 요청하고 WAV를 base64 data URL로 묶어 JSON으로 반환한다.

3. 발견된 문제점

#	심각도	분류	위치(경로:줄)	내용
1	높음	보안	api/routers/llm.py:253,376,506,596,639,750,1287,1578,1726	LLM 10개 엔드포인트 전부 인증 의존성(Depends) 없음. 누구나 서버의 Anthropic/OpenAI/OpenRouter 키로 무제한 토큰 소비 가능
2	높음	보안	api/routers/billing.py:83-119	Stripe 웹훅이 서명(stripe-signature) 검증 없이 JSON 본문을 신뢰. 임의 checkout.session.completed POST로 plan_id=business, credits_remaining=-1 부여 가능
3	높음	보안	supabase/migrations/002_user_plans_and_persona.sql:23-25	user_plans에 for all using (auth.uid() = user_id) 정책 -> 로그인 사용자가 anon 키 클라이언트로 자기 plan_id를 business로 UPDATE 가능(권한 상승)
4	높음	보안	api/routers/tts.py:59-71	label을 그대로 파일명으로 사용(../.. 미차단), 확장자 검증·크기 제한·인증 없음 -> 경로 조작으로 서버 임의 위치에 파일 쓰기
5	높음	보안	api/routers/export.py:224-228, export.py:195-200	cover_url을 urllib.request.urlretrieve로 서버가 직접 가져옴(SSRF). WeasyPrint가 사용자 HTML의 외부/file:// 리소스를 fetch 가능. 내보내기 5개 엔드포인트(:343,355,376,465,518) 모두 무인증
6	높음	보안	frontend/src/lib/admin.ts:17, hooks/use-is-pro.ts:13,35-41	Pro 잠금해제 비밀번호 "1234"가 클라이언트 번들에 포함, localStorage['flora_pro_unlocked']만으로 Pro 판정. 백엔드는 플랜 제한을 전혀 강제하지 않음
7	높음	설정	.github/workflows/deploy.yml:46,52	step 레벨 if: ${{ secrets.RAILWAY_TOKEN != '' }} — GitHub Actions는 step if에서 secrets 컨텍스트를 허용하지 않아 워크플로 자체가 유효하지 않음
8	높음	설정	api/Dockerfile:1-12, api/routers/export.py:134-139, api/.dockerignore	python:3.11-slim에 WeasyPrint 시스템 라이브러리(pango/cairo)와 CJK 폰트가 없음 -> 컨테이너에서 WeasyPrint import 실패·한글 PDF 깨짐. 폰트 경로는 C:/Windows/Fonts/malgun.ttf 하드코딩. .dockerignore에 .venv311/·ref_audio/·server.log 누락으로 이미지 비대
9	높음	성능	api/routers/llm.py:117,153,167,181, cover.py:173,201,224,249, export.py:199,493, f5-tts/api_server.py:116	async def 핸들러 안에서 동기 requests.post(timeout 120s)·동기 SDK·WeasyPrint 렌더·subprocess.run·tts.infer를 호출 -> 이벤트 루프 블로킹. 동시 사용자 2명부터 서로 대기
10	높음	버그	frontend/src/app/api/books/transform/route.ts:22-35 + api/routers/llm.py:245-262	"AI 고도화"가 instruction/title/genre를 보내지만 백엔드 WritingRequest에 해당 필드가 없어 무시되고, refine은 body.content[-1000:]만 처리 -> 전체 원고 고도화가 마지막 1,000자만 다듬고 끝남

#	심각도	분류	위치(경로:줄)	내용
11	중간	보안	frontend/src/app/api/books/build-html/route.ts:46	markdownToHtml이 제목만 escape하고 일반 단락은 escape 없이 <p>에 삽입 -> LLM/사용자 원고의 HTML이 그대로 미리보기 iframe(new/ebook/page.tsx:3953)과 PDF에 주입
12	중간	보안	frontend/.env:3,16, frontend/.env.local:3	프론트엔드 디렉터리 .env에 SUPABASE_SERVICE_ROLE_KEY와 OPENROUTER_API_KEY 실제 값 존재. 현재는 코드에서 참조하지 않지만 Vercel 환경변수로 그대로 올리면 유출 경로가 됨
13	중간	보안	api/routers/auth.py:17-18, cover.py:20-21	SUPABASE_URL 미설정이면 JWT 검증 없이 dev-user 반환. 운영 환경변수 누락 시 전원 동일 계정으로 통과
14	중간	보안	supabase/migrations/004_fix_constraints_and_api_keys.sql:19-27, api/routers/settings.py:16-19	사용자 API 키를 평문 컬럼에 저장하도록 설계하고 GET이 행 전체를 반환. 실제로는 프론트가 키를 localStorage(settings/page.tsx:201)에만 저장하고 db.get_user_api_keys(db.py:165)는 아무 곳에서도 호출되지 않는 죽은 기능
15	중간	버그	api/routers/waitlist.py:4 + api/main.py:36	prefix 중복으로 실제 경로 /api/api/waitlist. 프론트에서 waitlist를 호출하는 코드가 없고 in-memory 리스트 + 시드 카운트 147(:8)로 가짜 수치 반환
16	중간	버그	api/db.py:46, db.py:85-88	.single()은 행이 없으면 예외를 던지므로 get_project가 404 대신 500. delete_project는 Supabase 모드에서 삭제 여부와 무관하게 True
17	중간	설정	api/requirements.txt vs llm.py:69,179, cover.py:166	requests, google-genai(코드는 from google import genai; 명시된 google-generativeai는 다른 패키지), soundfile, stripe 미기재 -> 배포 시 ImportError가 광범위한 except Exception에 흡수되어 조용히 mock 응답
18	중간	설정	railway.toml:3, api/Dockerfile:5	빌드 컨텍스트가 저장소 루트이면 COPY requirements.txt .가 실패. Railway 서비스 Root Directory를 api/로 지정하지 않으면 배포 불가
19	중간	설정	.github/workflows/ci.yml:58, start-dev.ps1:8	CI는 Python 3.12, 프로젝트 표준은 3.11.9. 개발 스크립트는 venv가 아닌 전역 python을 호출(CLAUDE.md가 명시한 장애 원인)하고 F5-TTS 서버는 띄우지 않음
20	중간	설정	f5-tts/Dockerfile:19-30	업스트림 F5-TTS Dockerfile 그대로(GitHub clone, 포트 7860, api_server.py 미포함) -> FLORA TTS 서버 배포용으로 쓸 수 없음
21	중간	보안	f5-tts/api_server.py:92-93	ref_audio가 존재하는 로컬 경로이면 그대로 읽음 -> 호출자가 서버 파일 시스템의 임의 파일을 레퍼런스로 지정 가능
22	중간	성능	api/routers/tts.py:128-130,177-195,206	전체 WAV를 base64 data URL로 JSON에 포함(원본의 1.33배 메모리), 멀티보이스 세그먼트를 순차 호출, 본문 8,000자에서 조용히 절단
23	중간	구조	frontend/src/app/(dashboard)/dashboard/projects/new/ebook/page.tsx (4,095줄), manuscript/page.tsx (1,422줄)	단일 페이지에 useState 68개·fetch 12곳. 원고 업로드 페이지가 제목추천/기획안/집필/메타데이터 호출(manuscript/page.tsx:449-634)을 그대로 복제

#	심각도	분류	위치(경로:줄)	내용
24	중간	구조	api/routers/cover.py:17-55 vs auth.py:12-52, billing.py:9	토큰 검증 로직이 두 파일에 복제되어 있고 billing이 cover에서 auth를 import. 요청마다 create_client로 Supabase 클라이언트 재생성
25	중간	품질	api/routers/llm.py (16곳), cover.py, export.py:201,230	except Exception 후 mock 반환이 광범위 -> 운영 장애가 사용자에게 "정상 응답(mock=true)"으로 보임. 로깅 없음
26	낮음	버그	api/routers/export.py:62	ePUB 식별자가 abs(hash(title)) — 프로세스마다 달라져 동일 책의 ID가 매번 바뀜
27	낮음	품질	supabase/migrations/001_initial_schema.sql:6	alter database postgres set "app.jwt_secret" to 'your-jwt-secret' 플레이스홀더가 마이그레이션에 남아 있음
28	낮음	구조	f5-tts/src/f5_tts/train/, eval/, runtime/triton_trtllm/ 하위	학습·평가·Triton 런타임 코드까지 벤더링. 필요한 것은 pip install f5-tts + 158줄 래퍼
29	낮음	품질	CLAUDE.md = frontend/CLAUDE.md (동일 내용), frontend/-w, frontend/the author's persona.json, api/server.log	문서 중복, 잡파일(HTML 덤프, 로그)이 소스 트리에 존재
30	낮음	성능	frontend/src/hooks/use-auto-save.ts:52	JSON.stringify(data)를 의존성 배열에 넣어 매 렌더마다 전체 상태 직렬화

심각도 높음 항목 상세

#1 LLM 엔드포인트 무인증. llm.py 전체에 Depends(verify_token)이 한 곳도 없다. /api/llm/write-section은 target_length 10,000자 기준으로 토큰을 산정해 Claude/GPT-4o를 호출하므로 호출 1건당 비용이 크다. CORS는 origin만 제한하므로 curl 공격을 막지 못한다. llm.py:1750의 /debug-api-keys는 어떤 키가 설정돼 있는지까지 공개한다.

#2 Stripe 웹훅 서명 미검증. WebhookEvent(BaseModel)로 본문을 파싱하고 client_reference_id를 그대로 user_id로 사용한다. 공격자는 자기 user_id를 넣은 가짜 이벤트로 business 플랜과 무제한 크레딧을 얻는다. stripe.Webhook.construct_event(payload, sig, STRIPE_WEBHOOK_SECRET) 검증이 필수이며, 플랜 변경은 Stripe 세션을 서버에서 재조회한 뒤에만 반영해야 한다.

#3 user_plans RLS 과다 허용. 정책이 FOR ALL이므로 SELECT뿐 아니라 UPDATE/INSERT도 본인 행에 허용된다. 프론트는 anon 키로 Supabase 클라이언트를 만들기 때문에 로그인 사용자는 브라우저 콘솔에서 supabase.from('user_plans').upsert({plan_id:'business', credits_remaining:-1})를 실행할 수 있다. 플랜 테이블은 SELECT만 허용하고 쓰기는 service role(백엔드)만 하도록 바꿔야 한다.

#4 레퍼런스 WAV 업로드 경로 조작. safe_name = (label or file.filename).replace(" ", "_")만 수행하므로 label="../../main"이 통과한다. 쓰기 위치는 api/ref_audio 상위이며 .wav만 덧붙인다. 파일명은 UUID로 서버가 생성하고, 매직바이트로 WAV를 검증하고, 크기를 제한해야 한다.

#5 내보내기 SSRF. cover_url이 http로 시작하면 서버가 어떤 URL이든 다운로드한다(클라우드 메타데이터 엔드포인트 포함). WeasyPrint 역시 <img src>·@import 등으로 내부망 요청을 만들 수 있다. 허용 호스트 화이트리스트와 WeasyPrint url_fetcher 제한이 필요하다.

#6 Pro 게이팅이 클라이언트 전용. useIsPro가 true면 어조 슬라이더·캔버스·오디오북 UI가 열리지만, 백엔드 tts.py·llm.py는 플랜을 전혀 확인하지 않는다. 즉 UI 잠금은 장식에 가깝고 수익 모델이 코드로 보호되지 않는다.

#7~#8 배포 설정 비정상. deploy.yml은 문법 오류로 실행되지 않고, api/Dockerfile로 빌드한 이미지는 WeasyPrint가 import되지 않아 항상 fpdf2 폴백으로 내려가며 한글 폰트가 없어 PDF가 깨진다. 결과적으로 "Vercel + Railway" 배포는 현재 상태로는 성립하지 않는다.

#9 동기 블로킹. Uvicorn 단일 워커에서 `async def` 핸들러가 120초 동기 HTTP 호출을 하면 해당 시간 동안 `/health` 포함 모든 요청이 멈춘다. `asyncio.to_thread` 또는 비동기 SDK(`AsyncAnthropic`, `AsyncOpenAI`, `httpx.AsyncClient`)로 전환해야 한다.

#10 AI 고도화 기능 오동작. 11단계 파이프라인의 "10단계 AI 고도화"는 Next Route Handler가 백엔드 `refine`을 프록시하는데, 백엔드는 `selection`이 없으면 `content[-1000:]`만 사용한다. 수만 자 원고를 보내도 마지막 1,000자만 돌아오며, 프론트는 그 결과를 원고 전체로 교체할 위험이 있다.

4. 효율성 개선 제안

구조/아키텍처

- 인증 의존성을 `api/deps.py` 하나로 통합(`verify_token` + 캐시된 Supabase 클라이언트 + 플랜 조회)하고 모든 라우터에 일괄 적용. `optional_verify`(cover.py) 삭제.
- `llm.py` 1,759줄을 `services/llm_client.py`(제공자 라우팅), `routers/llm_plan.py`(기획·목차), `routers/llm_write.py`(집필), `data/mock_*.py`(mock 데이터 1,000줄 이상)로 분리.
- 프론트 `new/ebook/page.tsx`를 단계별 컴포넌트(`steps/Step01Genre.tsx` …)와 `useEbookWizard` 리듀서로 분해. `manuscript/page.tsx`는 같은 스텝 컴포넌트를 재사용하도록 통합.
- LLM/TTS/내보내기 호출을 `lib/api.ts`로 모아 토큰 주입·에러 처리·타입을 한 곳에서 관리.
- Pro 판정과 플랜 제한값을 백엔드 단일 소스(`plans` 테이블)에서 내려받고, 프론트는 표시만 담당.

성능

- 외부 호출을 비동기 클라이언트로 교체하고, PDF 렌더(WeasyPrint/fpdf2/Typst)와 F5-TTS 추론은 `run_in_threadpool`로 격리.
- TTS 결과는 Supabase Storage에 업로드 후 서명 URL 반환(JSON base64 제거). 멀티보이스 세그먼트는 `asyncio.gather`로 병렬화(동시성 2~3 제한).
- 긴 작업(전체 집필, 오디오북 생성)은 작업 테이블 + 폴링 또는 SSE로 전환해 Vercel/Railway 요청 타임아웃을 회피.
- `use-auto-save`의 직렬화 의존성 제거, debounce 유지.

개발 생산성

- `requirements.txt`를 실제 import 기준으로 재생성(`pip-compile`), `pyproject.toml`로 통일. CI Python 버전을 3.11로 맞춤.
- `f5-tts/` 벤더링을 제거하고 `pip install f5-tts==1.1.20` + `api_server.py`만 유지. `ckpts`는 볼륨/HF 캐시로.
- 백엔드 최소 테스트: 인증 실패 401, 웹훅 서명 실패 400, 경로 조작 거부, waitlist 경로. 프론트는 `build-html` escape 테스트.
- CLAUDE.md 중복 제거(frontend는 `@../CLAUDE.md` 참조만), 잡파일(`-w`, `server.log`, `tsbuildinfo`) 삭제 및 `.gitignore` 추가.

운영/배포

- `api/Dockerfile`: `apt-get install libpango-1.0-0 libpangoft2-1.0-0 libcairo2 fonts-noto-cjk` 추가, `.dockerignore`에 `venv`·로그·`ref_audio` 추가, Railway Root Directory `api` 설정 또는 `railway.toml`을 `api/`로 이동.
- `deploy.yml`의 `secrets` 조건을 `env`로 옮기고 job 레벨에서 분기. Stripe 웹훅 시크릿·CORS 운영 도메인·`SUPABASE_JWT_SECRET`(또는 JWKS) 환경변수 체크리스트 작성.
- F5-TTS는 GPU 호스트 별도 배포가 필요하므로 `f5-tts/api_server.py` 전용 Dockerfile(CUDA 베이스 + `pip install f5-tts soundfile`) 작성 또는 외부 TTS API로 대체 검토.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	백엔드 인증 통합: api/deps.py 생성(verify_token 단일화, Supabase 클라이언트 lru_cache, mock 모드는 ENV=dev에서만 허용), llm.py·tts.py·export.py·billing.py·cover.py 전 엔드포인트에 Depends 적용, /debug-api-keys 삭제	P0	M	Fable 5.1	인증/권한 로직, 다중 모듈 횡단 변경, 실수 시 비용 유출
2	Stripe 웹훅 서명 검증(construct_event + STRIPE_WEBHOOK_SECRET), 세션 재조회 후 플랜 반영, user_plans RLS를 SELECT 전용으로 변경하는 마이그레이션 005 작성	P0	S	Fable 5.1	결제·권한 상승 차단, 데이터 모델 정책 변경
3	파일/URL 입력 하드닝: tts.py upload-ref UUID 파일명·WAV 매직바이트·10MB 제한, export.py _resolve_cover_image 허용 호스트 화이트리스트, WeasyPrint url_fetcher 제한, api_server.py 로컬 경로 입력 차단	P0	M	Fable 5.1	경로 조작·SSRF 보안 수정
4	동기 호출 비동기화: call_llm을 AsyncAnthropic/AsyncOpenAI/httpx.AsyncClient로 교체, cover.py requests -> httpx async, PDF 렌더·Typst·F5 추론은 run_in_threadpool	P0	M	Fable 5.1	성능 병목 진단과 LLM 라우팅 로직 재작성이 얽힘
5	배포 설정 복구: api/Dockerfile에 pango/cairo/Noto CJK 설치, .dockerignore 보완, export.py 폰트 탐색을 OS 무관하게 변경, railway.toml 경로 정합, deploy.yml secrets 조건 수정, ci.yml Python 3.11	P0	M	Opus 5.5	명확히 정의된 설정 수정이지만 다수 파일·검증 필요
6	requirements.txt 재생성(requests, google-genai, soundfile, stripe, bs4 포함) 및 except Exception 16곳에 logging.exception 추가, mock 폴백은 dev 모드에서만	P1	S	Sonnet 5.5	설정 파일·보일러플레이트 정리
7	AI 고도화 파이프라인 수정: 백엔드에 /api/llm/transform(청크 분할 + instruction 수용) 신설, transform/route.ts 연결, build-html/route.ts:46 단락 escape 추가	P1	M	Opus 5.5	2~3개 파일에 걸친 중간 규모 기능 수정
8	플랜 제한 서버 강제: plans.max_projects·크레딧 차감을 projects.py·llm.py·tts.py에서 검사, useIsPro에서 비밀번호/localStorage 우회 제거, PasswordUnlockModal dev 빌드 한정	P1	M	Fable 5.1	결제 모델 보호 로직, 백엔드·프론트 동시 변경
9	new/ebook/page.tsx(4,095줄) 단계별 컴포넌트 10개 + useEbookWizard 리듀서로 분해, manuscript/page.tsx가 동일 스텝 재사용	P1	L	Fable 5.1	다중 모듈 리팩토링, 상태 흐름 재설계
10	lib/api.ts 확장: llmApi/ttsApi/exportApi 추가(토큰 자동 주입, 타입), 페이지의 raw fetch 20곳 교체	P1	M	Opus 5.5	상태관리·API 연동 정리
11	TTS 산출물 Supabase Storage 저장 + 서명 URL 반환, 멀티보이스 asyncio.gather(세마포어 3), 8,000자 절단을 청크 순차 생성으로 대체	P1	M	Opus 5.5	명확한 API 연동·중간 난이도
12	waitlist 경로 수정(/waitlist prefix) + Supabase waitlist 테이블 마이그레이션 + 랜딩 페이지 연결, 시드 카운트 제거	P2	S	Sonnet 5.5	단순 버그·보일러플레이트
13	db.py 오류 처리(maybe_single, delete 결과 확인), ePUB uid를 uuid5로, datetime.utcnow 교체	P2	S	Sonnet 5.5	단순 버그 수정
14	f5-tts/ 벤더링 제거 -> pip install f5-tts + api_server.py 전용 Dockerfile(CUDA) 작성, start_api.bat/start-dev.ps1을 venv 경로 기반으로 통일하고 3개 서버 모두 기동	P2	M	Opus 5.5	환경 구성 작업, 검증 필요

#	작업	우선순위	예상규모	권장 모델	선정 이유
15	문서·잡파일 정리: frontend/CLAUDE.md 참조로 축소, -w·server.log·tsbuildinfo 삭제 및 .gitignore 보강, frontend/.env에서 서비스 롤 키·OpenRouter 키 제거	P2	S	Sonnet 5.5	문서화·설정 정리

6. 종합 평가

강점

1. 전자책 제작 파이프라인이 실제로 끝까지 동작한다. 기획안·목차·집필·메타데이터 LLM 호출, 다중 제공자 폴백(_resolve_llm, OpenRouter 후보 순회), 7종 표지 제공자, WeasyPrint/fpdf2/Typst 3중 PDF 경로가 모두 코드로 존재하며 로그로 검증된다.
2. 출판 도메인 지식이 코드에 반영되어 있다. 5대 판형 프리셋(안쪽 여백 > 바깥 여백), @page 규칙, RFC 5987 한글 파일명, ePUB3 nav 구조, 옴니버스 모드, 60인 페르소나 데이터 등은 다시 만들기 어려운 자산이다.
3. F5-TTS 로컬 서버 래퍼가 Windows torchaudio 문제까지 해결한 상태로 동작하고, 멀티보이스 합성(화자 분리 -> 세그먼트 TTS -> 무음 삽입 -> concat)이 구현되어 있다.

약점

1. 백엔드가 "데모 모드"다. LLM·TTS·내보내기 무인증, 웹훅 미검증, RLS 과다 허용, 클라이언트 전용 Pro 게이팅으로 비용·권한·수익 모두 보호되지 않는다.
2. 배포가 성립하지 않는다. Dockerfile 시스템 의존성 누락, deploy.yml 문법 오류, requirements 누락, Windows 폰트 경로 하드코딩이 겹쳐 "로컬에서만 되는" 상태다.
3. 프론트 유지보수성이 임계점이다. 4,095줄 페이지(useState 68개)와 1,422줄 복제 페이지, 1,473줄 위저드가 있어 기능 하나를 바꾸면 회귀 범위를 예측하기 어렵다.

모델별 작업 비율

- Fable 5.1: 6건 (40%) — 인증 통합, 웹훅/RLS, 입력 하드닝, 비동기화, 플랜 강제, 페이지 분해
- Opus 5.5: 5건 (33%) — 배포 설정, AI 고도화 수정, api.ts 정리, TTS 저장, F5 환경
- Sonnet 5.5: 4건 (27%) — requirements/로깅, waitlist, db 버그, 문서 정리

권장 진행 순서

1. 1단계(보안 봉합, 1주): 작업 1·2·3 -> 인증 전면 적용, 웹훅 검증, RLS 수정, 파일/URL 하드닝. 이 단계 전에는 공개 URL로 배포하면 안 된다.
2. 2단계(동작 보장, 1~2주): 작업 4·5·6·7 -> 비동기화, Dockerfile/CI 복구, 의존성 정리, AI 고도화 수정. 이 시점에 Railway/Vercel 첫 배포.
3. 3단계(구조 개선, 2~3주): 작업 8·9·10·11 -> 플랜 서버 강제, 페이지 분해, API 클라이언트 통합, TTS 스토리지. 이후 12~15는 틈틈이.

통합 판단 (flora vs Flora-v2 vs flora-publisher)

flora는 세 저장소 중 기능적으로 가장 많이 완성되어 있지만 프론트 구조 부채가 가장 크다. Flora-v2는 구조와 기획 문서가 우수하나 엔드투엔드로 동작하지 않는 골격이다. 권고는 "Flora-v2를 본선으로 유지하고 flora의 검증된 엔진을 이식한 뒤 flora를 archive/flora-v1로 보존"이다. 이식 대상은 api/routers/llm.py의 call_llm·_call_via_openrouter(비동기화해서), cover.py 제공자 7종, export.py의 WeasyPrint/Typst/ePUB 빌더, frontend/src/lib/print-presets.ts·typst-presets.ts·html-templates.ts, app/api/books/build-html 라우트, f5-tts/api_server.py 계약(/v1/speech, gen_text), lib/personas.ts 데이터다. 상세 근거는 Flora-v2.md 6장에 정리했다.

주 담당 모델: Fable 5.1

이 프로젝트의 남은 일은 "기능 추가"가 아니라 인증·결제·RLS·SSRF를 한 번에 봉합하고, 동기 블로킹을 비동기 구조로 바꾸고, 4,000줄 페이지를 해체하는 횡단 리팩토링이다. 세 작업 모두 모듈 경계를 다시 긋는 설계 판단이 필요하고, 실수 시 비용(API 키 남용, 무료 Pro 전환, 데이터 접근)이 크다. 따라서 1~2단계는 Fable 5.1이 주도하고, 설정·보일러플레이트·문서는 Sonnet 5.5로 내려 비용을 절약하는 배치가 적절하다.